

PYTHON SELENIUM AUTOMATION

PROJECT REPORT

Submitted By

Srijita Ghosh

Section: A

Roll No.: 42

Enrollment No.: 12023002028043

B.Tech

Computer Science & Engineering
(Artificial Intelligence & Machine Learning)

Academic Project Report

Certificate

This is to certify that Srijita Ghosh, Section A, Roll No. 42, Enrollment No. 12023002028043, a student of B.Tech Computer Science & Engineering (Artificial Intelligence & Machine Learning), has completed the project report entitled “Python Selenium Automation” as part of the academic curriculum.

The project demonstrates the application of Python-based Selenium WebDriver automation for testing key functionalities of a web-based e-commerce application.

Faculty/Guide Name: _____

Department: _____

Institution: _____

Date: _____

Signature: _____

Declaration

I, Srijita Ghosh, student of B.Tech Computer Science & Engineering (Artificial Intelligence & Machine Learning), Section A, Roll No. 42, Enrollment No. 12023002028043, hereby declare that the project report entitled “Python Selenium Automation” has been prepared for academic purposes.

The report describes the design and implementation of a Python-based Selenium automation framework for testing selected functionalities of the TutorialsNinja Demo e-commerce website.

I have prepared this report based on the project implementation, source files, configuration files, test scripts, and supporting project materials.

Student Name: Srijita Ghosh

Signature: _____

Date: _____

Acknowledgement

I would like to express my sincere gratitude to my faculty members and mentors for their guidance and support throughout the development of this project.

I am thankful to my department and institution for providing the academic environment and resources required to understand software testing and automation concepts.

I would also like to acknowledge the technical learning materials and project resources that helped me understand Python, Selenium WebDriver, PyTest, Page Object Model, test-data management, logging, screenshots, and automated reporting.

Finally, I would like to thank everyone who provided guidance and encouragement during the development and documentation of this project.

Srijita Ghosh

Table of Contents

1. Abstract
2. Introduction
3. Selenium and Web Automation
4. Application Under Test
5. Problem Statement
6. Objectives
7. Scope of the Project
8. Technologies and Tools Used
9. System Requirements
10. Selenium Architecture
11. Project Framework Architecture
12. Page Object Model
13. Configuration Management
14. CSV Test Data
15. PyTest Configuration and Test Discovery
16. Selenium Locators
17. WebDriver Operations
18. Wait Management
19. Assertions
20. Browser Test
21. Login Test
22. Product Search Test
23. Automatic Failure Screenshots
24. Logging
25. Test Cases
26. Test Execution and Reporting
27. Results and Analysis
28. Security Considerations
29. Version Control and Project Cleanliness
30. UnitTest and PyTest
31. Advantages
32. Limitations

33. Challenges

34. Future Scope

35. Conclusion

36. References

37. Viva Questions and Answers

1. Abstract

Web applications require regular testing to ensure that their functionality works correctly and consistently. Manual testing can become repetitive and time-consuming when the same scenarios need to be executed repeatedly. Automation testing provides a way to execute predefined test scenarios using software tools and programming frameworks.

This project presents a Python Selenium Automation Framework developed for testing the TutorialsNinja Demo e-commerce website. The framework uses Selenium WebDriver for browser automation and PyTest for test execution. The framework follows the Page Object Model (POM) design pattern and includes reusable utilities for configuration management, CSV-based test data, logging, screenshots, and HTML reporting.

The implemented test suite covers website/browser verification, login testing using CSV-based credentials, and product search testing using the product name “iPhone.” The framework also provides automatic screenshots when a test fails, logging functionality, configurable browser/application settings, and HTML test reporting.

The project demonstrates how a structured automation framework can organize browser interactions, test logic, reusable utilities, test data, and reporting into separate components.

2. Introduction

2.1 Web Automation

Web automation is the process of using software tools to automatically interact with websites and web applications. Instead of a tester manually performing every browser operation, an automation script can open the browser, navigate to a URL, interact with elements, submit information, and validate results.

Web automation is particularly useful for repetitive and repeatable test scenarios. It can be used for functional testing, regression testing, smoke testing, and validation of common user workflows.

2.2 Manual Testing versus Automation Testing

Manual Testing	Automation Testing
Performed manually by a tester	Performed using automation scripts
Repetitive tasks take more time	Repetitive tasks can be executed automatically
Manual execution can vary	Predefined steps and assertions provide consistency
Evidence may need manual collection	Screenshots and reports can be generated automatically
Large regression suites are time-consuming to repeat	Automated suites can be rerun efficiently

2.3 Python with Selenium

Python is a readable and widely used programming language for automation. Selenium provides the browser automation interface, while Python provides the programming environment in which test cases, page objects, utilities, and assertions are implemented.

2.4 Importance of Automated Testing

- Speed of repeated test execution
- Repeatability of predefined scenarios
- Consistent validation through assertions
- Support for regression testing
- Reduced repetitive manual effort
- Ability to organize larger test suites into reusable components

3. Selenium and Web Automation

3.1 Selenium

Selenium is a web automation framework used to automate interactions with web browsers. It allows test scripts to open websites, locate web elements, enter text, click controls, navigate through pages, and validate application behavior.

3.2 Selenium WebDriver

Selenium WebDriver provides programmatic control over a web browser. The project uses Selenium WebDriver as the core browser-automation component.

3.3 Selenium Ecosystem

The Selenium ecosystem includes tools and APIs for browser automation. Selenium WebDriver is the component directly relevant to this project because the framework drives a real browser while executing its test cases.

3.4 Web Elements

Web elements are individual components of a web page such as text boxes, buttons, links, menus, search fields, and product listings. Selenium locators are used to identify these elements so that the automation script can interact with them.

4. Application Under Test

The application tested by the framework is the TutorialsNinja Demo e-commerce website.

Item	Value
Website	TutorialsNinja Demo
URL	https://tutorialsninja.com/demo/
Application Type	Demo e-commerce website

The project framework is designed around browser-based testing of this application and its selected e-commerce functionality.

5. Problem Statement

Testing an e-commerce application manually involves repeatedly performing browser interactions such as opening the application, entering login information, searching for products, and checking whether expected results appear. Repeating these tasks can consume time and introduce inconsistency in manual execution.

The project addresses this problem by implementing automated test cases using Python, Selenium WebDriver, and PyTest. Reusable page objects, configuration management, external test data, logging, screenshots, and reporting are used to organize the automation process.

6. Objectives

1. Automate web browser interactions using Selenium WebDriver.
2. Develop automated test cases using Python.
3. Use PyTest for test execution and automatic test discovery.
4. Validate selected functionality of the TutorialsNinja Demo website.
5. Automate login functionality.
6. Automate product search functionality.
7. Apply the Page Object Model to organize page-level functionality.
8. Separate login test data from the test implementation using CSV.
9. Implement reusable configuration management.
10. Provide logging for automation activities.
11. Capture screenshots when a test fails.
12. Support HTML test reporting.

7. Scope of the Project

The current implementation focuses on selected automated scenarios of the TutorialsNinja Demo e-commerce website.

- Opening and validating the website
- Valid login testing
- Product search testing
- Chrome browser automation in the supplied browser setup
- CSV-based login test data
- Configuration management
- PyTest test execution
- Automatic failure screenshots
- Logging
- HTML reporting

The project can be expanded in the future to automate additional e-commerce workflows such as product details, cart operations, checkout, account management, and broader browser coverage.

8. Technologies and Tools Used

Technology / Tool	Purpose	Project Evidence
-------------------	---------	------------------

Python	Programming language	Used by all automation scripts and utilities
Selenium WebDriver	Browser automation	Core browser automation technology
PyTest	Test execution	Main test framework
PyTest HTML	HTML reporting	Dependency and reporting feature
Page Object Model	Framework design	Used by login and product-search page classes
CSV	Test data	Used for login credentials
ConfigParser	Configuration management	Used by ConfigReader
Python Logging	Logging	Creates console and file logging
Chrome	Browser	Used in supplied unittest browser setup

9. System Requirements

9.1 Hardware Requirements

Exact hardware specifications were not specified in the supplied project files. A computer capable of running Python, a modern web browser, Selenium WebDriver, and PyTest is required.

9.2 Software Requirements

Package / Component	Version / Requirement
Selenium	4.49.0
PyTest	9.1.1
PyTest HTML	4.2.0
Pandas	3.0.5
Browser	Chrome in supplied browser setup
Python	Exact project version not specified in supplied files
Operating System	Not specified in supplied files

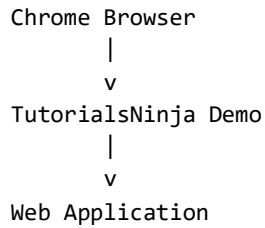
The exact package versions above are taken from the supplied requirements.txt.

10. Selenium Architecture

```

Python Test Script
|
v
Selenium WebDriver
|
v

```



The Python test script defines the required actions and assertions. Selenium WebDriver provides browser control. The browser executes those operations against the TutorialsNinja application, after which the test evaluates the resulting state using assertions.

11. Project Framework Architecture

```

Selenium_Ecommerce_Framework/
├── .gitignore
├── conftest.py
├── pytest.ini
├── README.md
├── requirements.txt
├── base/
│   ├── __init__.py
│   └── base_test.py
├── config/
│   └── config.ini
├── pages/
│   ├── __init__.py
│   ├── login_page.py
│   └── product_search_page.py
├── reports/
│   ├── automation.log
│   └── test_report.html
├── screenshots/
├── test_data/
│   └── login_data.csv
├── tests/
│   ├── __init__.py
│   ├── test_browser.py
│   ├── test_login.py
│   └── test_product_search.py
├── utilities/
│   ├── __init__.py
│   ├── config_reader.py
│   ├── csv_reader.py
│   ├── logger.py
│   └── screenshot.py
  
```

```
|
└─ venv/
```

Component	Responsibility
base/	Common test/base functionality
config/	Configuration values
pages/	Page Object classes
tests/	Automated test cases
utilities/	Reusable helper classes
test_data/	External test data
reports/	Logs and HTML test report
screenshots/	Failure screenshots
conftest.py	PyTest hook for failure handling
pytest.ini	PyTest discovery/execution configuration
requirements.txt	Python package dependencies

12. Page Object Model

The Page Object Model is a test-automation design pattern in which page-specific behavior is represented by dedicated classes. The project contains LoginPage and ProductSearchPage classes, while the test files focus on test scenarios and assertions.

```
TestLogin
|
v
LoginPage
|
v
TutorialsNinja Login Functionality

TestProductSearch
|
v
ProductSearchPage
|
v
TutorialsNinja Search Functionality
```

This separation improves organization because browser interaction logic can remain in page classes while the test classes describe the scenarios being validated.

13. Configuration Management

The framework includes a ConfigReader utility that reads configuration values from config/config.ini. It uses Python's configparser module and dynamically determines the project root before constructing the configuration path.

```

import configparser
import os

class ConfigReader:
    @staticmethod
    def get_config():
        config = configparser.ConfigParser()
        # determine project root
        # construct config/config.ini path
        # read and return configuration
        return config

```

The supplied unittest browser implementation retrieves the browser, base URL, and implicit wait from the configuration. This keeps these settings outside the test logic.

14. CSV Test Data

The login test uses an external CSV file to supply login information. CSVReader locates test_data/login_data.csv, opens it using UTF-8 encoding, and uses csv.DictReader to convert rows into dictionaries.

```

test_data = CSVReader.read_login_data()

email = test_data[0]["email"]
password = test_data[0]["password"]

```

The login test then passes these values to the LoginPage object. Separating data from test logic makes the test easier to adapt to additional data records.

The project's .gitignore explicitly excludes the login data file under its test-credentials section, helping prevent test credentials from being committed to version control.

15. PyTest Configuration and Test Discovery

```

[pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
addopts = -v

```

The configuration tells PyTest to search the tests directory, recognize test_*.py files, recognize Test* classes, recognize test_* functions, and use verbose output.

This supports automatic discovery of the project's browser, login, and product-search test scripts.

16. Selenium Locators

Selenium requires a way to identify web elements before interacting with them. Common locator strategies include ID, Name, Class Name, Tag Name, Link Text, Partial Link Text, XPath, and CSS Selector.

Locator	Description	Typical Syntax
---------	-------------	----------------

ID	Finds an element by its HTML id attribute	By.ID, "element_id"
Name	Finds an element by name attribute	By.NAME, "element_name"
Class Name	Finds an element by class	By.CLASS_NAME, "class_name"
Tag Name	Finds an HTML tag	By.TAG_NAME, "input"
Link Text	Finds a link by visible text	By.LINK_TEXT, "Login"
Partial Link Text	Finds part of link text	By.PARTIAL_LINK_TEXT, "Log"
XPath	Uses an XPath expression	By.XPATH, "//input[@id='x']"
CSS Selector	Uses a CSS selector	By.CSS_SELECTOR, "#x"

The supplied test files delegate page-level interaction to LoginPage and ProductSearchPage. The exact locator definitions inside those page classes were not included in the supplied source excerpts, so specific project locators are not asserted here.

17. WebDriver Operations

Operation	Purpose	Project Context
webdriver.Chrome()	Creates a Chrome WebDriver session	Used in supplied unittest browser setup
driver.get(url)	Opens a URL	Used to open the configured base URL
driver.title	Reads current page title	Used for browser/title validation
driver.current_url	Reads current URL	Printed by login/product-search tests
driver.quit()	Ends the WebDriver session	Used during cleanup
driver.save_screenshot()	Captures browser screenshot	Used by ScreenshotUtil

```
options = Options()
options.add_argument("--start-maximized")

self.driver = webdriver.Chrome(options=options)

self.driver.get(base_url)
```

18. Wait Management

Web applications may load elements asynchronously. Waits help automation synchronize browser operations with application behavior.

18.1 Implicit Wait

An implicit wait applies a general waiting period when WebDriver searches for elements. The supplied unittest browser test reads the wait value from configuration and applies it with `driver.implicitly_wait()`.

```
self.driver.implicitly_wait(
    implicit_wait
)
```

18.2 Explicit Wait

An explicit wait is targeted at a particular condition, such as element visibility or clickability. The project README identifies explicit waits as a framework feature, but the exact explicit-wait implementation was not included in the supplied source files.

19. Assertions

Assertions compare an observed application state with an expected condition. They are central to automated test validation.

```
assert "Your Store" in self.driver.title
```

The browser test uses this assertion to verify the expected title.

```
assert login_page.is_login_successful(), "Login was not successful"
```

The login test validates the result through the LoginPage page object.

```
assert product_search_page.are_products_displayed(), "No products were displayed in
search results"
```

The product-search test validates that products are displayed after searching.

20. Browser Test

The browser test is implemented in `test_browser.py` and inherits from `BaseTest`.

```
from base.base_test import BaseTest

class TestBrowser(BaseTest):

    def test_open_website(self):
        assert "Your Store" in self.driver.title
```

The test checks whether the expected text “Your Store” is present in the browser title. This provides a basic application-opening validation.

21. Login Test

```
from base.base_test import BaseTest
from pages.login_page import LoginPage
from utilities.csv_reader import CSVReader

class TestLogin(BaseTest):
```

```

def test_valid_login(self):
    test_data = CSVReader.read_login_data()

    email = test_data[0]["email"]
    password = test_data[0]["password"]

    login_page = LoginPage(self.driver)

    login_page.login(email, password)

    print("Current URL:", self.driver.current_url)
    print("Page Title:", self.driver.title)

    assert login_page.is_login_successful(), "Login was not successful"

```

The test reads the login data, extracts the first record, creates a LoginPage object, performs the login, prints browser information, and validates the result using an assertion.

22. Product Search Test

```

from base.base_test import BaseTest
from pages.product_search_page import ProductSearchPage

class TestProductSearch(BaseTest):

    def test_product_search(self):
        product_search_page = ProductSearchPage(self.driver)

        product_search_page.search_product("iPhone")

        print("Current URL:", self.driver.current_url)
        print("Page Title:", self.driver.title)

        print(
            "Search Results Heading:",
            product_search_page.get_search_results_heading()
        )

        assert product_search_page.are_products_displayed(), "No products were displayed in search results"

```

The test searches for “iPhone,” records URL and page-title information, retrieves the search-results heading, and validates that products are displayed.

23. Automatic Failure Screenshots

The framework uses a PyTest hook in conftest.py to capture screenshots when a test fails.

```

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()

    if report.when != "call":

```

```

return

if report.failed:
    # verify test instance and WebDriver
    # create safe screenshot name
    # capture screenshot while browser is open

```

The hook processes only the actual test-call result rather than setup or teardown results. It verifies that the test instance and WebDriver exist before requesting a screenshot.

ScreenshotUtil creates the screenshots directory if necessary, generates a timestamp, builds a PNG filename, and calls `driver.save_screenshot()`.

24. Logging

The Logger utility provides reusable logging for the framework. It creates a reports directory if needed and writes to `reports/automation.log`.

```

logging.INFO

reports/automation.log

%(asctime)s - %(levelname)s - %(name)s - %(message)s

```

The logger configures both a FileHandler and a StreamHandler, allowing messages to be written to a file and displayed in the console. It also checks for existing handlers to avoid duplicate handlers.

25. Test Cases

ID	Scenario	Precondition	Expected Result	Validation
TC-01	Open TutorialsNinja website	Browser/application available	Page title contains “Your Store”	Title assertion
TC-02	Valid login	Login data available	Login is successful	LoginPage.is_login_successful()
TC-03	Search for iPhone	Application opened	Products are displayed	ProductSearchPage.are_products_displayed()

These scenarios are derived from the supplied `test_browser.py`, `test_login.py`, and `test_product_search.py` files.

26. Test Execution and Reporting

26.1 PyTest Execution

The project's PyTest configuration enables verbose output and automatic discovery. A standard execution command is:

```
pytest
```

26.2 Dependencies

```
selenium==4.49.0
pytest==9.1.1
pytest-html==4.2.0
pandas==3.0.5
```

26.3 HTML Reporting

The project includes PyTest HTML as a dependency and defines reports/test_report.html in its documented project structure. HTML reporting provides a browser-readable summary of automated test execution.

26.4 Failure Evidence

When a test fails during its call phase and a WebDriver is available, the PyTest hook can capture a screenshot. This creates visual evidence that can assist debugging.

27. Results and Analysis

The supplied source materials identify the test scenarios and reporting mechanisms, but they do not provide a complete final execution-output record containing the final total number of passed, failed, skipped tests and execution duration. Therefore, these values are not fabricated.

Metric	Result
Total Test Cases	Not specified in supplied execution output
Passed	Not specified
Failed	Not specified
Skipped	Not specified
Execution Time	Not specified

The implementation itself provides the mechanisms required to execute the tests, validate expected results, record logs, capture failure screenshots, and produce HTML reports.

28. Security Considerations

The login test reads credentials from an external CSV file instead of embedding them directly in the test method.

The project's .gitignore excludes test_data/login_data.csv under a test-credentials section. This is useful for preventing local credential data from being tracked in version control.

For production environments, credentials should be managed through secure secret-management mechanisms rather than plaintext files.

29. Version Control and Project Cleanliness

The supplied .gitignore excludes virtual-environment files, Python cache files, PyTest cache, generated HTML reports, log files, screenshots, IDE settings, operating-system files, packaging directories, and test credentials.

```

venv/
__pycache__/
.pytest_cache/
reports/*.html
reports/*.log
screenshots/*.png
.vscode/
test_data/login_data.csv

```

These exclusions help keep generated artifacts, local environments, and sensitive test data separate from source-controlled project files.

30. unittest and PyTest

The project also contains unittest_browser_test.py, which demonstrates browser automation using Python's built-in unittest framework.

```

class TestUnittestBrowser(unittest.TestCase):

    def setUp(self):
        config = ConfigReader.get_config()
        browser = config["DEFAULT"]["browser"]
        base_url = config["DEFAULT"]["base_url"]
        implicit_wait = int(
            config["DEFAULT"]["implicit_wait"]
        )

        # Start Chrome and open application

    def tearDown(self):
        if hasattr(self, "driver"):
            self.driver.quit()

```

The unittest example configures Chrome, applies an implicit wait, opens the application, and verifies the page title using self.assertIn(). It provides an additional demonstration of browser automation, while the main framework uses PyTest.

31. Advantages

- Structured automation through separate test, page, base, and utility components
- Reusable page objects and utilities
- Automated test discovery with PyTest

- External CSV-based test data
- Configurable browser and application settings
- Failure screenshots for debugging
- File and console logging
- HTML reporting
- Clear project organization
- Reduced repetition for recurring test scenarios

32. Limitations

- Dynamic web elements may require locator or synchronization maintenance.
- Changes to the application's HTML structure can affect page objects.
- CAPTCHA mechanisms can prevent straightforward automation.
- Browser automation depends on application and network availability.
- Locators must be maintained when the UI changes.
- Authentication flows may require specialized handling.

These are general Selenium automation considerations. The supplied files do not identify each item as a specific failure encountered during this project.

33. Challenges

The framework addresses several common automation-development concerns through its architecture. These include configuration management, test-data handling, synchronization, reusable page objects, failure evidence, logging, and reporting.

The supplied source files do not contain a complete development diary of specific problems encountered, so individual issues are not presented as confirmed historical events.

34. Future Scope

- Automate additional e-commerce workflows such as cart and checkout.
- Expand data-driven testing to multiple test records and scenarios.
- Add cross-browser execution.
- Introduce parallel test execution for larger suites.
- Integrate the framework with CI/CD pipelines.
- Use richer reporting and historical test dashboards.
- Extend browser execution using Selenium Grid or cloud testing services.
- Use secure environment variables or secret-management tools for credentials.
- Containerize the framework for consistent execution environments.

35. Conclusion

The Python Selenium Automation project demonstrates the development of a structured browser-automation framework for the TutorialsNinja Demo e-commerce website. The framework combines Python, Selenium WebDriver, and PyTest to automate selected application scenarios.

The implementation uses the Page Object Model to separate page-level behavior from test logic and includes reusable utilities for configuration management, CSV test data, logging, screenshots, and reporting.

The implemented scenarios cover website-title verification, valid login, and product search for “iPhone.” The framework also demonstrates assertions, test discovery, implicit-wait configuration, failure screenshot handling, logging, and HTML reporting.

The project provides a practical foundation for maintainable web test automation and can be extended with additional test cases, browsers, execution environments, reporting capabilities, and secure test-data management.

36. References

13. Project README and framework documentation supplied with the project.
14. Python Selenium source files supplied with the project.
15. PyTest configuration and test scripts supplied with the project.
16. Configuration, CSV-reader, logging, screenshot, and failure-hook utilities supplied with the project.
17. requirements.txt supplied with the project.
18. Selenium documentation: <https://www.selenium.dev/documentation/>
19. Python documentation: <https://docs.python.org/>
20. PyTest documentation: <https://docs.pytest.org/>

37. Viva Questions and Answers

Q1. What is Selenium?

Answer: Selenium is a framework used to automate web browsers and test web applications.

Q2. What is Selenium WebDriver?

Answer: It provides programmatic control over a web browser.

Q3. Which programming language is used in this project?

Answer: Python.

Q4. Which website is tested?

Answer: TutorialsNinja Demo.

Q5. Which testing framework is used?

Answer: PyTest.

Q6. What design pattern is used?

Answer: Page Object Model (POM).

Q7. What is POM?

Answer: A design pattern that separates page-specific browser interactions from test logic.

Q8. Why is POM useful?

Answer: It improves organization, reuse, and maintainability.

Q9. What is PyTest?

Answer: A Python testing framework used to write and execute automated tests.

Q10. What is an assertion?

Answer: A check that verifies whether an expected condition is true.

Q11. What does the browser test verify?

Answer: It verifies that “Your Store” is present in the browser title.

Q12. What is the purpose of driver.get()?

Answer: It navigates the browser to a specified URL.

Q13. What is driver.quit()?

Answer: It ends the WebDriver session and closes the browser.

Q14. What is an implicit wait?

Answer: A general wait applied when WebDriver searches for elements.

Q15. What is an explicit wait?

Answer: A wait for a particular condition to become true.

Q16. Why is CSV used in the project?

Answer: To keep login test data separate from test logic.

Q17. What does CSVReader do?

Answer: It reads login data from test_data/login_data.csv.

Q18. What does ConfigReader do?

Answer: It reads configuration from config/config.ini.

Q19. What does conftest.py do?

Answer: It contains a PyTest hook used to capture screenshots on test failure.

Q20. When is the screenshot hook triggered?

Answer: When the actual test call fails and a WebDriver is available.

Q21. Where are screenshots stored?

Answer: In the screenshots directory.

Q22. What does ScreenshotUtil do?

Answer: It creates and saves browser screenshots.

Q23. What does Logger do?

Answer: It provides reusable console and file logging.

Appendix A – Key Project Files

File	Role
test_browser.py	Verifies application title
test_login.py	Executes valid-login scenario
test_product_search.py	Executes product-search scenario
conftest.py	PyTest failure screenshot hook
pytest.ini	PyTest discovery configuration
config_reader.py	Reads config.ini
csv_reader.py	Reads login CSV data
logger.py	Creates file and console logging
screenshot.py	Captures timestamped screenshots
requirements.txt	Defines package versions
README.md	Documents project overview, technologies, features, and structure